Rattrapage Programmation C++

12/03/2026 - Durée : 1h00 - Documents papiers autorisés.

Attention : Vous veillerez à écrire du code le plus propre possible respectant les règles présentées en cours. Notamment, vous ferez attention à nommer explicitement vos variables, vos fonctions ou vos méthodes, à utiliser à bon escient les références, à utiliser correctement le mot clé « const » dès que nécessaire, à respecter les principes SOLID, à vérifier la validité des données utilisées lors de la création des objets, …

Exercice 1 : Modélisation d'un cours

On suppose que la classe Student représentant un étudiant est fournie.

student.h

```
class Student {
private:
    int id_;
    std::string firstname_;
    std::string lastname_;

public:
    Student(int id, const std::string& firstname, const std::string& lastname);
    int id() const;
    const std::string& firstname() const;
    const std::string& lastname() const;
};
```

On souhaite modéliser un système simple de gestion de cours universitaires. Un cours (`class Course`) est caractérisé par un titre (type `std::string`, non vide), une durée en heures (type `int`, strictement positif), un nombre maximal d'étudiants pouvant s'inscrire au cours (type `int`, strictement positif) et une liste des étudiants inscrits (type `std::vector<Student>`, vide à la création d'un cours).

Q1 - Déclaration de la classe `Course`: Écrire la déclaration de la classe `Course`, incluant les attributs, le constructeur et les getters.

Q2 - Constructeur et getters : Écrire le code du constructeur et des getters de `Course`.

Q3 - Nombre maximal de participants atteint : Écrire le code de la méthode `isFull` qui renvoie true si le nombre d'étudiants inscrits a atteint le maximum, et false sinon.

Q4 - Recherche d'étudiant inscrit : Écrire le code de la méthode `findStudent` qui cherche un étudiant à partir de son identifiant dans le vecteur. La méthode retourne la position de l'étudiant dans le vecteur des inscrits si l'étudiant est présent ou retourne -1 si l'étudiant n'est pas inscrit.

Q5 - Inscription d'un étudiant : Écrire le code de la méthode `registerStudent` qui inscrit un étudiant à un cours uniquement s'il n'est pas déjà présent et si le cours n'a pas atteint son maximum d'étudiants. La méthode indiquera si l'ajout a bien été effectué en retournant `true` et `false` sinon.

Q6 - Suppression d'un étudiant : Écrire le code de la méthode `removeStudent` qui cherche un étudiant à partir de son identifiant et l'enlève du vecteur des inscrits d'un cours uniquement s'il est présent. Utiliser la méthode `iterator erase(iterator pos)` de la STL pour supprimer un élément du vecteur. La méthode `removeStudent` indiquera si la suppression a bien été effectuée en retournant `true` et `false` si l'étudiant n'a pas été trouvé dans le vecteur.

Q7 - Tri des étudiants : Écrire le code de la méthode `sortStudents` qui trie le vecteur d'étudiants par ordre alphabétique de leur nom (lastame), puis par leur prénom (firstname) si besoin. Utiliser la méthode

`std::sort(iterator first, iterator last, lambda)` de la STL, avec l'expression lambda de comparaison suivante `[](const Student& a, const Student& b)` pour effectuer le tri.

Q8 - Création d'une classe pour les cours en ligne : On souhaite maintenant modéliser un cours en ligne à l'aide d'une classe `OnlineCourse` dérivée de `Course`. Déclarer cette nouvelle classe en ajoutant un attribut nommé plateforme (std ::string, non vide) correspondant à la plateforme technologique utilisée pour les cours en ligne (par exemple, « Teams », « Zoom », …) et le getter associé.

Q9 - Constructeur et getter : Écrire le code du constructeur et du getter de `OnlineCourse`.

Q10 - Information sur le cours : On souhaite obtenir des informations détaillées sur les cours réalisés sous la forme « Course : <title> - Duration : <duration>h - Students : <number> (<max>) » dans laquelle <title>, <duration>, <number>, <max> sont remplacés respectivement par le titre du cours, la durée du cours, le nombre d'inscrits et le nombre maximum d'étudiants

Ecrire le code de la méthode `info` permettant de retourner ces informations sous la forme d'une chaine de caractères.

Surcharger la méthode `info` dans la classe `OnlineCourse` afin de pouvoir également obtenir les informations sur la plateforme utilisée : Course : <title> - Duration : <duration>h - Students : <number> (<max>) - Platform <platform> ».